In [14]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import lightgbm as lgb
```

In [15]:
```
df = pd.read_csv("uber.csv")
```

In [16]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 9 columns):
 #   Column             Non-Null Count   Dtype  
---  ------             --------------   -----  
 0   Unnamed: 0         200000 non-null  int64  
 1   key                200000 non-null  object 
 2   fare_amount        200000 non-null  float64
 3   pickup_datetime    200000 non-null  object 
 4   pickup_longitude   200000 non-null  float64
 5   pickup_latitude    200000 non-null  float64
 6   dropoff_longitude  199999 non-null  float64
 7   dropoff_latitude   199999 non-null  float64
 8   passenger_count    200000 non-null  int64  
dtypes: float64(5), int64(2), object(2)
memory usage: 13.7+ MB
```

In [17]:
```
df.dropna(how="any", inplace=True)
df.isnull().sum()
```

Out[17]:
```
Unnamed: 0           0
key                  0
fare_amount          0
pickup_datetime      0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    0
dropoff_latitude     0
passenger_count      0
dtype: int64
```

In [18]:
```
df = df[
    (df.pickup_latitude > -90) & (df.pickup_latitude < 90) &
    (df.dropoff_latitude > -90) & (df.dropoff_latitude < 90) &
    (df.pickup_longitude > -180) & (df.pickup_longitude < 180) &
    (df.dropoff_longitude > -180) & (df.dropoff_longitude < 180) &
    (df.fare_amount > 0) & (df.passenger_count > 0) & (df.passenger_count < 50
]
```

In [19]:
```
def distance(lat1, lon1, lat2, lon2):
    lon1, lon2, lat1, lat2 = map(np.radians, [lon1, lon2, lat1, lat2])
    diff_lon = lon2 - lon1
    diff_lat = lat2 - lat1
```

```
    km = 2 * 6371 * np.arcsin(np.sqrt(np.sin(diff_lat / 2.0) ** 2 +
                                      np.cos(lat1) * np.cos(lat2) * np.sin(dif
    return km

df["distance_km"] = distance(df.pickup_latitude, df.pickup_longitude,
                             df.dropoff_latitude, df.dropoff_longitude)
```

In [20]:
```
df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"], format="%Y-%m-%d

df["is_weekend"] = df["pickup_datetime"].dt.dayofweek >= 5
df["day_type"] = df["is_weekend"].astype(int)

df["hour"] = df["pickup_datetime"].dt.hour
df["dayofweek"] = df["pickup_datetime"].dt.dayofweek
df["month"] = df["pickup_datetime"].dt.month
df["year"] = df["pickup_datetime"].dt.year

df["dlon"] = df.pickup_longitude - df.dropoff_longitude
df["dlat"] = df.pickup_latitude - df.dropoff_latitude

def time_of_day(hour):
    if 6 <= hour < 12:
        return "morning"
    elif 12 <= hour < 18:
        return "midday"
    else:
        return "night"

df["time_of_day_label"] = df["pickup_datetime"].dt.hour.apply(time_of_day)
df["time_of_day"] = df["time_of_day_label"].map({"morning": 0, "midday": 1, "r
```

In [21]:
```
df = df[df["distance_km"] > 0.2]

for col in ["pickup_latitude", "dropoff_latitude"]:
    df = df[df[col].between(40.4, 41.2)]
for col in ["pickup_longitude", "dropoff_longitude"]:
    df = df[df[col].between(-74.5, -73.3)]

df = df[df["fare_amount"] <= 25]

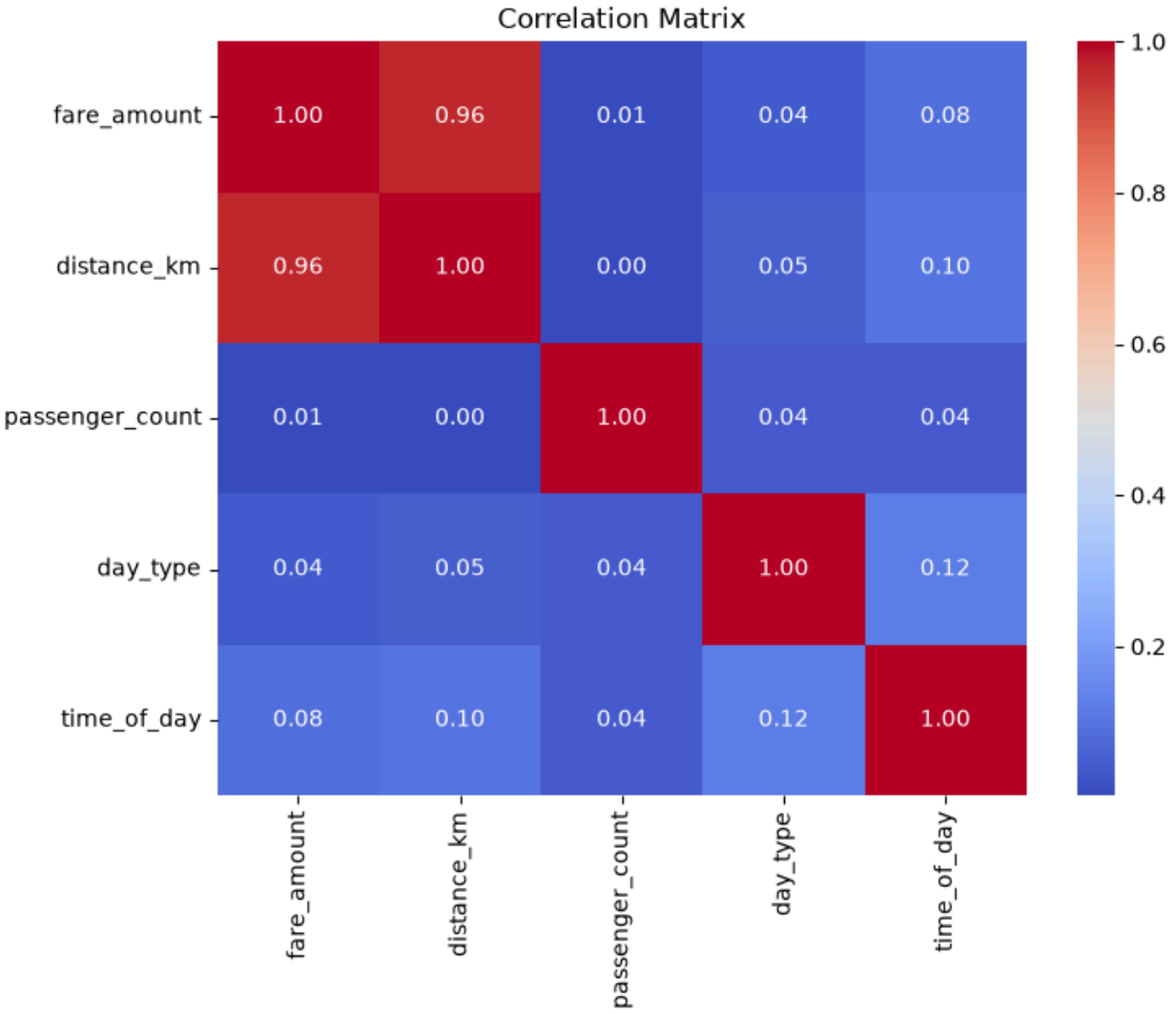
A = np.vstack([df["distance_km"].values, np.ones(len(df))]).T
coef, *_ = np.linalg.lstsq(A, df["fare_amount"].values, rcond=None)
slope, intercept = coef
resid = np.abs(df["fare_amount"].values - (intercept + slope * df["distance_km
df = df[resid <= 1.75]

print("rows after cleaning:", len(df))
```

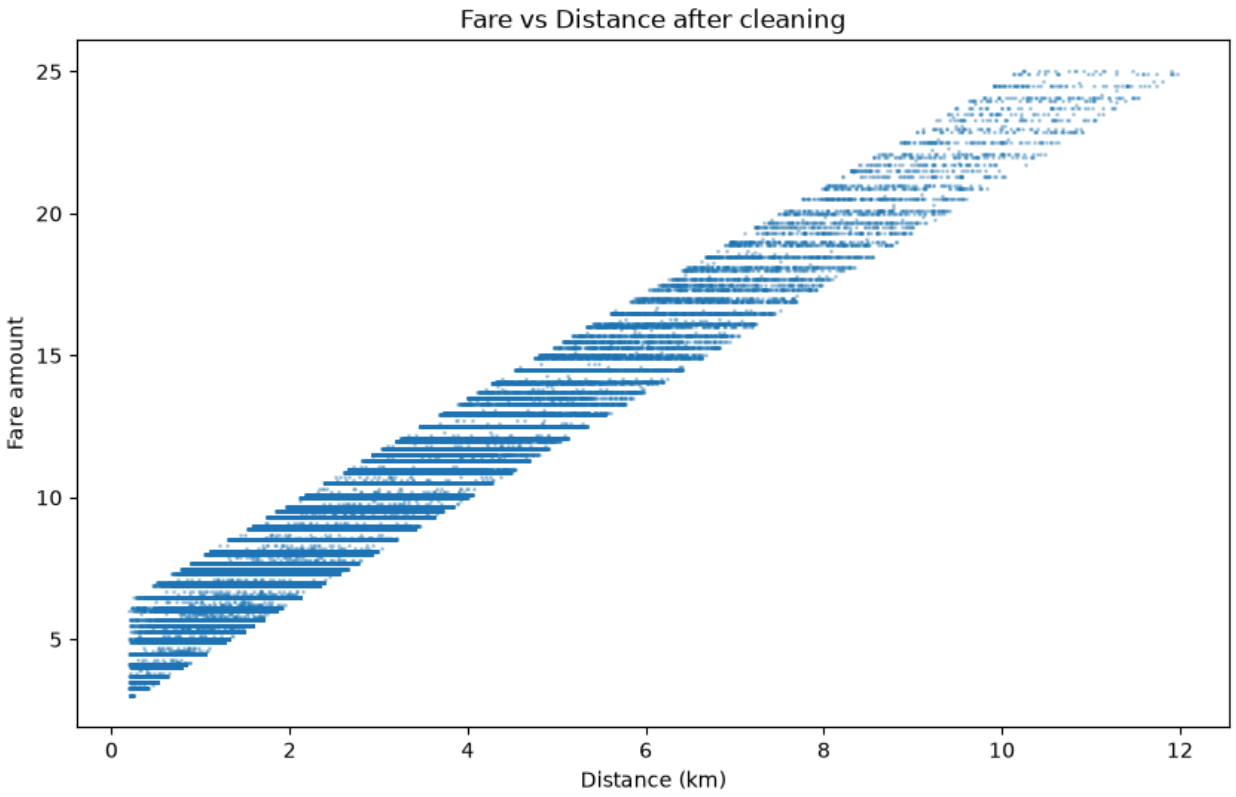
```
rows after cleaning: 113634
```

In [22]:
```
corr_cols = ["fare_amount", "distance_km", "passenger_count", "day_type", "tim
plt.figure(figsize=(8, 6))
sns.heatmap(df[corr_cols].corr(), annot=True, cmap="coolwarm", fmt=".2f")
```

```
plt.title("Correlation Matrix")
plt.show()
```



In [23]:
```
plt.figure(figsize=(10, 6))
plt.scatter(df["distance_km"], df["fare_amount"], s=1, alpha=0.3)
plt.xlabel("Distance (km)")
plt.ylabel("Fare amount")
plt.title("Fare vs Distance after cleaning")
plt.show()
```



In [24]:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

X = df[["distance_km", "dlon", "dlat", "hour", "dayofweek", "month", "year", "
y = df["fare_amount"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

lr = LinearRegression().fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

rf = RandomForestRegressor(n_estimators=200, min_samples_leaf=2, random_state=
y_pred_rf = rf.predict(X_test)


for name, y_pred in [("Linear Regression", y_pred_lr), ("Random Forest", y_pre
    print(f"\n{name}")
    print(f"  R2:   {r2_score(y_test, y_pred):.4f}")
    print(f"  MAE:  {mean_absolute_error(y_test, y_pred):.4f}")
    print(f"  RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.4f}")
```

```
Linear Regression
  R2:   0.9319
  MAE:  0.7502
  RMSE: 0.8968

Random Forest
  R2:   0.9390
  MAE:  0.6975
  RMSE: 0.8491
```